

# CSC2044 Concurrent Programming

## Group Assignment

### Concurrent Bank Account Transaction System

#### **Group 5**

Qhuisten Ng - 23130370

Choong Zhen Yang - 23029465

Ahnaf Shabab Kabir – 24017550

# Task 1: System design

This bank account transaction system was created to satisfy the requirements of the assignment. It is made up of four main components found at different levels of the program. First, we will look at the Bank account class, of which there are three different classes. The first class (i.e. BankAccount) is used in Task 2 and Task 3. The other two classes, named UnsafeBankAccount and SafeBankAccount are used for Task 5 and Task 6 respectively. The first class is a general class while the latter two are specialized classes and are missing some of the former's features. These classes all have a single shared variable called balance and multiple class methods used to mimic different bank transactions.

Second is the transaction methods. The general class has four different methods: deposit(int amount), withdraw(int amount), getBalance() and serviceTax(). They mimic different real-life bank transactions. The first two methods allow for the deposit and withdrawal of money from a bank account class object by the amount indicated by the input variable. The third method simply returns the current value of the balance variable. The last method applies a service charge to the account, essentially withdrawing a fixed amount of money from the account object. The last method is unavailable in the two specialized classes.

Third, transaction threads are the thread objects used to perform each transaction when done so concurrently. Therefore, they are only unused in the Task 2 section of the program. These threads are created as needed within the runnable main class java file during the program operation.

The fourth and last component is the balance variable of the bank account class. This is a shared variable and is considered sensitive information. For a successful transaction system, the value of this variable must be modified and stored accurately. The different tasks will show how concurrency can negatively affect the handling of this variable, leading to incorrect values unless steps are taken to prevent the issue.

| Component                | Description                                                                                                                                                               |
|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Bank account classes     | Represents a real-life bank account. There are three different classes in this program.                                                                                   |
| Transaction methods      | Methods mimicking typical bank account transactions. Similarly named transactions have been designed differently in different classes as needed for each task             |
| Transaction threads      | Individual threads created within the main java file that perform concurrent transactions on a bank account                                                               |
| Account balance variable | The important, shared variable that is read and written by methods performed by different threads simultaneously. Requires synchronization to be consistent and accurate. |

## Task 2: Sequential Transaction Processing

This form of transaction processing is described as sequential because no two transactions happen at the same time. They are sequenced one after another, where one finishes, the print statement notifies the user, and the next one begins.

## Task 3: Concurrent Transaction Processing Using Threads

In the concurrent version of the bank account transaction system, there is an object “private final Object lock = new Object(); ”, which protects the balance from being

updating by different threads at the same time. The lock object is used in these methods, `deposit()`, `withdraw()`, `checkBalance()` and `serviceTax()`.

In comparison to sequential programming, concurrent programming's output will look differently most of the time as the output order changes at each run, the reason the order change is because of how thread scheduler manage the execution each time. Modern systems use pre-emptive scheduling, which the scheduler gets to decide which thread gets CPU time during when and for how long. The scheduler can take CPU time away from one thread and give it to another, which cause the execution and outputs of different thread to become interleave. Output orders often vary between different runs due to dynamic factors like CPU availability, operating system decisions, background processes, timing differences, and Java Virtual Machine (JVM) implementation.

Calling `start()` creates a new thread because this specific method instructs the JVM to construct and launch a new, independent path of execution. Creating a thread object (like `new MyThread()`) only creates the object in memory but calling `start()` is what begins the life of the new thread, transitioning it to the `RUNNABLE` state so the scheduler can allocate CPU time to it.

Calling `run()` directly does not create a new thread because `run()` is simply the method containing the instructions the thread is meant to execute. If you call it directly, it behaves exactly like an ordinary method call. The code will execute sequentially inside whichever thread called it (such as the main thread) rather than running concurrently in a new child thread

## Task 4: Shared Data, Stack, and Heap Explanation

Which variables are local/private to a thread

Variables only found within the body of the thread lambda statement are local/private to a thread. They live on each thread's own call stack and are never visible to any other thread. For example, multiples threads running for-loops do not share their value of the loop counter. Even though four threads run at the same time, each one has its own copy of these values.

Which objects or variables are shared by multiple threads

The account object is being shared by all four threads, as they hold a reference to the same BankAccount instance. It means that when the balance is initialised at 500 it is the data that every thread will read and modify. The lock object is being shared as well, as it is essential for the synchronization to work across threads, with one thread locking it the dataframe to update the data at a time.

### Why local variables are usually safer in multithreaded programs

A local variable only exists in one thread stack frame. No other thread can reach it, so there is no need for locks or coordination. Safety is automatic. Shared objects, by contrast, live on the heap and are reachable by anyone holding a reference to them, which is exactly the situation with account.

### Why the shared bank account balance can create problems

In concurrency, the thread processes can read the balance and write the amount into the balance without knowing one another has updated it. For example, if one thread reads the balance 500 and another thread also read the balance as 500, then both threads compute the result, which leads to risks of overwriting one another. Hence the account has lost the track of one thread's activity. It is called race condition, which means the final value depends on which thread writes last and not in the correct sequence.

### How the shared balance is stored as part of the bank account object

Balance is an instance field on the BankAccount object, which lives on the heap as a shared item

## Task 5: Race Condition Demonstration

The program was made to calculate the final balance of an unsafe bank account after a few concurrent transactions. The account initially starts with a balance of RM500. Then it has a total of RM10000 deposited into it by a thread running a for-loop that adds RM1 to the account exactly 10000 times. Concurrently, RM5000 is withdrawn from the account in a

similar manner to the deposit process, a for-loop running thread that subtracts RM1 from the account exactly 5000 times.

The expected final value of the account balance is RM5500. As shown in the two output screenshots, found further down in the Task 7 section, the actual value calculated by the program is incorrect. The first value was 3754, lower than the expected outcome. The second value was 6258, higher than the expected outcome.

Both these results were incorrect due to the system relying on the uncontrolled ordering of operations to provide a correct output. This situation is called a “race condition”. Simply put, the correct answer requires that the different operations of the program be performed in a specific order, but the program does not include any control measures, so they are done randomly. This random order changes with every run of the program as well, hence the calculated value also changes every time.

The “balance = balance + amount” statement represents a combination of 'read' and 'write' operations being performed on the balance variable and is not a single atomic operation. The computer running this line of code performs the following simplified steps:

1. Read the value of balance from device memory
2. Perform the addition of the amount value to the balance value
3. Write the modified value of balance back into device memory

This problem is called race condition because it refers to different threads racing each other during each run of the program. When handling the combination of operations that make up a single line of code more than a certain amount of time (e.g. looping through “balance = balance + amount” several hundred times), the thread race will very rarely end in the order necessary to get the expected result. And since the result of the thread race also changes each run of the program, the calculated value does not even stay the same.

## Task 6: Synchronization Solution

The critical section of the code is the different methods of the Back Account class that represent different transactions, namely deposit, withdrawal, and balance check. To be more specific, it is the line of code that modifies the value of the balance variable. In this case, since the methods only contain one line of code, the entire method can be considered critical.

Each critical section should only be accessed by one thread so that each thread runs the operations involved in the critical code by itself completely before allowing another thread to start. This way, the balance variable is never outdated when read or incorrectly overwritten by a thread during 'read' and 'write' operations.

Ensuring singular access to critical sections is done by using synchronization, a built-in java language feature. By specifically tagging the critical code, or in this case the whole method, as synchronized, the JVM only allows one thread to access the tagged block. The problem of race condition is avoided as different threads are just not allowed to race against each other, instead only one thread is allowed on the racetrack (i.e. the critical code) and finish fully before another thread is allowed in.

As mentioned previously, limiting access to the critical section to one thread at a time protects the balance variable. In this concurrent bank account transaction system, the value of the balance variable is the shared heap data. So, to stop this data from being outdated when read or improperly overwritten by a thread, the synchronized tag only allows one thread to perform the 'read' and 'write' operations constituting the tagged critical code section.

## Task 7: Testing, Output Screenshots, and Reflection

### Reflection

1. What is the main difference between sequential and concurrent transaction processing?

In sequential processing, each transaction is separate, never overlapping with one another. A deposit transaction may occur for a bank account object, then a withdrawal, but they happen in separate time frames, not simultaneously. In contrast, concurrent processing entails the use of threads or runnable tasks within threads to execute two transactions within the same interval on a bank account object, meaning 2 processes are happening at the same time to the bank account. This speeds up transactions and mirrors the real-life application and use of these accounts by people.

2. What did you observe when multiple threads performed transactions?

They occur in an order that is not fixed, as different runs yield a different order of threads running the same set of transactions. These threads all have leverage over the bank account object and modify its balance, check the balance or apply a service charge. If they were synchronized, the output (final balance within the bank account) should be as expected regardless of the execution order, though the display method will show different values depending on where it is in the sequence.

### 3. Why is the output order not always fixed in concurrent programs?

Thread scheduling, which determines which threads get to run first, is dependent on operating system factors and the JVM (Java Virtual Machine) decisions.

### 4. What problem happened in the unsafe bank account program?

Two threads entered the critical section of their respective methods (deposit and withdraw) at the same time, causing both threads to read the bank account balance, perform computation and update the variable at the same time. Hence, the initial output is overwritten by the last thread to run, causing inconsistency in the data which piles up over many loops.

### 5. How did synchronization solve the problem?

It utilizes a lock object inherent to the instance object such that when the critical section of code is run by a thread, the thread obtains the lock to the object, allowing to complete fully and update the shared data before the next thread is allowed to execute. This ensures that the updated data is handled as expected between multiple threads.

### 6. What did this assignment teach you about developing safe concurrent Java applications?

If threads to facilitate concurrency are left to access shared data without control, race condition will occur and the resulting data will no longer be consistent and accurate. Hence, it is important that either the synchronized keyword be used on methods that will be used by multiple threads, or to lock only the critical section using a dedicated lock object in the class, reducing synchronization overhead and improving performance.



## Output Screenshots

### Sequential Transaction Processing

```
>>>>> Sequential Transaction Processing Output <<<<<
Initial account balance: RM500
```

```
Starting Transaction 1: Withdraw RM200
Finished Transaction 1
Current balance: RM300
```

```
Starting Transaction 2: Apply Service Charge
Finished Transaction 2
Current balance: RM295
```

```
Starting Transaction 3: Deposit RM1000
Finished Transaction 3
Current balance: RM1295
```

```
Starting Transaction 4: Apply Service Charge
Finished Transaction 4
Current balance: RM1290
```

```
Starting Transaction 5: Deposit RM300
Finished Transaction 5
Current balance: RM1590
```

```
Starting Transaction 6: Apply Service Charge
Finished Transaction 6
Current balance: RM1585
```

```
Starting Transaction 7: Withdraw RM600
Finished Transaction 7
Current balance: RM985
```

```
Starting Transaction 8: Apply Service Charge
Finished Transaction 8
Current balance: RM980
```

## Concurrent Transaction Processing

```
>>>> Concurrent Transaction Processing Output <<<<
Thread-0 started Deposit RM200
Thread-2 started Balance Check
Thread-1 started Withdraw RM100
Thread-3 started Service Charge
Thread-2 finished Balance Check
Thread-3 finished Service Charge
Thread-0 finished Deposit RM200
Thread-1 finished Withdraw RM100
```

```
>>>> Concurrent Transaction Processing Output <<<<
Thread-0 started Deposit RM200
Thread-1 started Withdraw RM100
Thread-2 started Balance Check
Thread-3 started Service Charge
Thread-2 finished Balance Check
Thread-3 finished Service Charge
Thread-1 finished Withdraw RM100
Thread-0 finished Deposit RM200
```

```
>>>> Concurrent Transaction Processing Output <<<<
Thread-0 started Deposit RM200
Thread-1 started Withdraw RM100
Thread-2 started Balance Check
Thread-3 started Service Charge
Thread-2 finished Balance Check
Thread-3 finished Service Charge
Thread-0 finished Deposit RM200
Thread-1 finished Withdraw RM100
```

## Race Condition Demonstration

```
>>>>> Race Condition Demonstration Output <<<<<
Expected balance: 5500
Calculated balance: 3754
```

```
>>>>> Race Condition Demonstration Output <<<<<
Expected balance: 5500
Calculated balance: 6258
```

## Synchronization Solution

```
>>>>> Synchronization Solution Output <<<<<
Expected balance: 5500
Calculated balance: 5500
```

## Group Contribution Table

| Student Name       | Student ID | Main responsibilities                                                                                                                                                                                     | Estimated Contribution |
|--------------------|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------|
| Qhuisten Ng        | 23130370   | <ul style="list-style-type: none"><li>Task 1: System Design</li><li>Task 2: Sequential Transaction Processing</li><li>Task 7: Reflection</li></ul>                                                        | 30%                    |
| Choong Zhen Yang   | 23029465   | <ul style="list-style-type: none"><li>Task 3: Concurrent Transaction Processing using Threads</li><li>Task 4: Shared Data, Stack, and Heap Explanation</li></ul>                                          | 30%                    |
| Ahnaf Shabab Kabir | 24017550   | <ul style="list-style-type: none"><li>Task 5: Race Condition Demonstration</li><li>Task 6: Synchronization Solution</li><li>Task 7: Testing &amp; Screenshot</li><li>README</li><li>Final Check</li></ul> | 40%                    |

